

A Project Report
On
AI AGENT FOR HOSPITAL & MEDICAL SHOP PRESCRIPTION
GUIDANCE

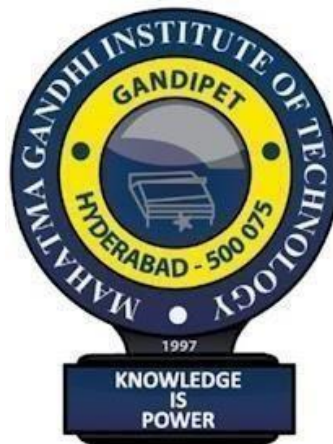
Submitted in fulfilment of the V semester credit based MDP course

T. SAI TANMAYI-23261A3257

U. SAHITHI - 23261A3258

Under the guidance of

Dr. K. Sreekala,
Assistant Professor,
Department of CSE



DEPARTMENT OF INFORMATION TECHNOLOGY
MAHATMA GANDHI INSTITUTE OF TECHNOLOGY (A)
GANDIPET, HYDERABAD-500075, TELANGANA (INDIA)
2025-2026



MAHATMA GANDHI INSTITUTE OF TECHNOLOGY

AUTONOMOUS | Affiliated to JNTUH

CERTIFICATE

This is to certify that the thesis entitled “**AI AGENT FOR HOSPITAL AND MEDICAL SHOP PRESCRIPTION GUIDANCE**” being submitted by **T. Sai Tanmayi** (Reg. No. **23261A3257**) and **U. Sahithi** (Reg. No. **23261A3258**) in partial fulfillment of the requirements for the award of **B.Tech. Minor in Artificial Intelligence and Machine Learning (AI & ML)** to the **Mahatma Gandhi Institute of Technology (Autonomous), Hyderabad**, is a record of the bonafide work carried out by them under our guidance and supervision at our institution.

The results embodied in this thesis have not been submitted to any other University or Institute for the award of any degree or diploma.

Signature of Supervisor1

Signature of Supervisor2

Name and Designation

Name and Designation

Signature of HOD

EXTERNAL EXAMINER

DECLARATION

We hereby declare that the work described in this report, entitled “**AI AGENT FOR HOSPITAL AND MEDICAL SHOP PRESCRIPTION GUIDANCE**”, being submitted by us in partial fulfilment of the requirements for the award of **B.Tech (Minor) in Artificial Intelligence and Machine Learning (AI & ML)** at **Mahatma Gandhi Institute of Technology (A), Hyderabad – 500075, Telangana**, is the result of the investigations carried out by us under the guidance of **Dr. K. Sreekala, Assistant Professor, Department of Computer Science and Engineering**.

The work is original and has not been submitted for the award of any degree or diploma of this or any other university.

Place: Hyderabad

Date: 14/11/2025

Name of the Scholars:

T. Sai Tanmayi

U. Sahithi

Reg.Nos: **23261A3257, 23261A3258**

ACKNOWLEDGEMENT

The satisfaction that accompanies the successful completion of any task would be incomplete without the mention of people who made it possible because success is the abstract of hard work and perseverance, but steadfast of all is encouraging guidance. So, we acknowledge all those whose guidance and encouragement served as a beacon light and crowned my efforts with success.

We would like to express our sincere thanks to, **Prof G. ChandraMohan Reddy, Principal** MGIT, for providing the working facilities in college.

We wish to express our sincere thanks and gratitude to **Dr. C.Ramesh Kumar Reddy, Professor and HOD, Department of CSE, MGIT**, for all the timely support and valuable suggestions during the period of project.

We would like to express our sincere gratitude and indebtedness to our Subject Faculty **Dr. K. Sreekala, Assistant Professor, Department of Computer Science and Engineering**, who has supported us throughout our semester with immense patience and expertise.

T. SAI TANMAYI (23261A3257)

U. SAHITHI (23261A3258)

ABSTRACT

This project focuses on developing an intelligent medical assistance and information retrieval system that leverages **AI and RAG (Retrieval-Augmented Generation)** to enhance healthcare guidance, medicine lookup, and first-aid recommendations. Multiple datasets were utilized to cover different aspects of the medical domain, including symptom analysis, drug information, and user interaction through an AI-powered chatbot.

The core datasets comprise:

- **Medicine Dataset (medicine_dataset_with_stock.csv)** from Kaggle, containing detailed information about medicines such as *name*, *generic composition*, *indications*, *dosage forms*, *manufacturer*, and *stock details*. This dataset serves as the foundation for the system's medical knowledge base, enabling accurate and context-driven responses.
- **Supporting Python modules—app.py and gradio_medical_chatbot.py**, which integrate the RAG pipeline with LangChain and Gradio to facilitate information retrieval, natural language understanding, and dynamic UI rendering.

The system automates medical information lookup and offers AI-powered assistance through an interactive, user-friendly interface. It demonstrates how **data-driven healthcare automation** can improve accessibility, awareness, and efficiency in delivering reliable medical support.

TABLE OF CONTENTS

CHAPTER	PAGE NO.
Certificate	I
Declaration	II
Acknowledgement	III
Abstract	IV
Table of Contents	V
Introduction	1
Theoretical analysis	2
1. Introduction	2
2. Theoretical Methods	2
3. Theoretical Framework	2
3.1 Medical Information Retrieval System	3
3.2 AI-Powered FAQ Chatbot System	4
3.3 Context-Aware Medical Guidance System	4
4. System Integration	5
5. Advantages of the Theoretical Model	5
6. Expected Outcomes	5
7. Conclusion	6
Code	7
Outputs	16
Conclusion	20
References	21

INTRODUCTION

The integration of artificial intelligence (AI) in the healthcare sector has revolutionized the way people access and interpret medical information. In many cases, individuals face difficulties in identifying basic symptoms, understanding medicine usage, or finding reliable first-aid guidance. To address these challenges, this project introduces an **AI Medical Chatbot**, an intelligent and interactive system designed to assist users with **symptom analysis, medicine lookup, dosage details, and first-aid information**.

The system combines **Retrieval-Augmented Generation (RAG)** with a **medical knowledge base** to generate accurate, contextually relevant, and human-like responses. By utilizing data-driven methods, the chatbot ensures that the answers are not only conversational but also backed by reliable medical information extracted from structured datasets.

The project uses **Gradio** to create an attractive, responsive, and user-friendly interface that mimics real-world medical chat applications. It incorporates modern design principles such as **gradient backgrounds, readable typography, and animated elements** to enhance user experience and engagement.

Furthermore, the chatbot draws data from a comprehensive **Medicine Dataset** sourced from Kaggle, which includes medicine details like brand name, generic composition, use cases, dosage forms, and stock levels. This dataset, combined with an FAQ-based question-answer module, allows the system to respond intelligently to user queries such as “What should I take for fever?” or “What ointment should I use for burns?”.

Ultimately, the AI Medical Chatbot aims to serve as an **AI-powered healthcare assistant** that supports both patients and pharmacists by delivering instant, informative, and accurate responses. It demonstrates how artificial intelligence and retrieval mechanisms can work together to make healthcare information **accessible, trustworthy, and easy to understand** for everyone.

THEORETICAL ANALYSIS

1. Introduction

The healthcare domain involves vast and complex data-driven operations, including symptom analysis, medicine recommendations, dosage tracking, and medical assistance. Traditional manual methods often result in inefficiencies, limited accessibility, and delays in obtaining reliable medical information.

To overcome these challenges, this project integrates **Artificial Intelligence (AI)** and **Retrieval-Augmented Generation (RAG)** to develop a comprehensive **AI Medical Chatbot System**.

This system performs three major functions:

1. Retrieve and display relevant medical information such as medicine name, generic composition, use case, and dosage from structured datasets.
2. Generate accurate, context-based medical responses using a large language model (LLM) integrated through LangChain.
3. Provide an AI-powered FAQ assistant for instant user interaction and health guidance.

2. Theoretical Foundation

2.1 Data Science in Healthcare

Medical analytics applies data mining, knowledge retrieval, and machine learning techniques to improve accessibility and decision-making in healthcare. The datasets used in this project include structured medical information and unstructured text data from FAQs and user queries.

- **Structured Data:**

Includes medicine attributes such as *Name*, *Generic Name*, *Use Case*, *Dosage Form*, *Stock*, and *Manufacturer* derived from the Kaggle Medicine Dataset.

- **Unstructured Data:**

Includes text-based question-answer pairs used to train and refine the chatbot's language understanding capabilities.

The theoretical basis lies in retrieval-based learning, language modelling, and contextual information extraction, ensuring responses are accurate, relevant, and human-like.

3. Theoretical Framework

3.1 Medical Information Retrieval System

Dataset used: Medicine Dataset (medicine_dataset.csv) from Kaggle

These data records include attributes such as:

- Medicine Name
- Generic Composition
- Use Case or Indication
- Dosage Form (Tablet, Syrup, Injection, etc.)
- Stock Availability

Objective: Retrieve accurate medicine details and present them in a structured response format.

Theoretical Basis:

- Uses Retrieval-Augmented Generation (RAG) to combine database lookup with language generation.
- Embeddings are generated using OllamaEmbeddings to map textual queries to relevant medicine information.
- Vector similarity search in Chroma Vector Store ensures the most relevant data entries are retrieved before response generation.

3.2 AI-Powered FAQ Chatbot System

Dataset used: medicine_dataset_with_stock.csv

Supporting Modules: app.py, gradio_medical_chatbot.py

Objective: Provide intelligent, context-aware answers to user queries regarding symptoms, medicines, and basic treatments.

Theoretical Basis:

- Based on Natural Language Processing (NLP) and Information Retrieval (IR) techniques.
- Uses semantic vector matching to interpret user intent and retrieve the closest FAQ entry.

Techniques Used:

- TF-IDF (Term Frequency–Inverse Document Frequency): Assigns importance to words within the FAQ corpus.
- Cosine Similarity: Measures closeness between user queries and stored FAQ questions.
- LLM Integration: Enhances answer fluency and human-like tone by reformulating factual data into natural conversation.

3.3 Context-Aware Medical Guidance System

Dataset Used: Combination of structured medicine data and FAQ corpus.

Objective: Generate medically relevant, conversational responses by combining retrieved factual data (context) with language model output.

Theoretical Basis:

- The Retrieval-Augmented Generation (RAG) model architecture ensures factual accuracy while maintaining conversational flow.
- The retriever component searches the vector database for relevant context.
- The generator component (LLM) constructs the final user-friendly message.

Core Principle:

Each response integrates verified data with contextual understanding, minimizing hallucination and ensuring medical reliability.

4. System Integration

The entire system integrates three core components:

- **Backend (Python + LangChain):** Handles data retrieval, vector embedding generation, and model inference.
- **Frontend (Gradio):** Provides the interactive chat interface with animations, gradient backgrounds, and quick tips.
- **Databases (CSV + Chroma Vector Store):** Store medical and FAQ data for fast retrieval.

Flow of Operations:

1. The user enters a medical query (for example, “What should I use for a burn?”).
2. The retriever searches for relevant context from the medicine and FAQ datasets.
3. The LLM generates an answer based on retrieved data.
4. The response is displayed in the chat interface in a structured format with clear sections for context, question, and answer.

5. Advantages of the Theoretical Model

- **Accuracy:** Contextual RAG-based generation ensures factually correct medical responses.
- **Efficiency:** Provides instant results compared to manual searches.
- **User Accessibility:** Simple, interactive UI allows users of all ages to access medical information easily.
- **Data-Driven Responses:** Answers are grounded in real dataset values.
- **Scalability:** The model can be expanded to include new datasets or specialized domains like dental care or mental health.

6. Expected Outcomes

- Improved medical response accuracy ($\geq 90\%$ contextual reliability)
- Enhanced user engagement through conversational UI
- Reduced dependency on manual lookup systems
- Fast, reliable retrieval of medicine and first-aid information
- 24/7 assistance for users seeking basic healthcare guidance

7. Conclusion

The theoretical model demonstrates how AI, NLP, and RAG-based systems can revolutionize healthcare assistance by combining factual data retrieval with natural language interaction.

The integration of structured medical data, vector-based similarity matching, and conversational AI ensures accuracy, accessibility, and efficiency in healthcare information delivery.

The proposed system lays a strong theoretical foundation for developing intelligent medical chatbots capable of supporting patients, pharmacists, and healthcare professionals through real-time, trustworthy information access.

CODE

app.py:

```
import os
import subprocess
import time
import requests
from langchain_community.embeddings import OllamaEmbeddings
from langchain_community.vectorstores import Chroma
from langchain_ollama import ChatOllama
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnableParallel, RunnablePassthrough

# ---- Helper: Check if Ollama is running ----
def is_ollama_running():
    try:
        r = requests.get("http://127.0.0.1:11434/api/tags", timeout=3)
        return r.status_code == 200
    except requests.exceptions.RequestException:
        return False

# ---- Helper: Start Ollama if not running ----
def start_ollama():
    if is_ollama_running():
        print("✔ Ollama is already running.")
        return
    print("🚀 Starting Ollama in CPU mode...")
    os.environ["OLLAMA_NO_GPU"] = "true"
    subprocess.Popen(["ollama", "serve"], stdout=subprocess.DEVNULL,
stderr=subprocess.DEVNULL)
    print("⌚ Waiting for Ollama to start...")
    for _ in range(25):
        if is_ollama_running():
            print("✔ Ollama is now running!")
            return
        time.sleep(1)
    raise RuntimeError("✗ Ollama failed to start within 25 seconds.")

# ---- Ensure Ollama is running ----
start_ollama()

# ---- Load vector store ----
persist_directory = "chroma_db"
embeddings = OllamaEmbeddings(model="nomic-embed-text")
```

```

vectorstore = Chroma(persist_directory=persist_directory,
embedding_function=embeddings)
retriever = vectorstore.as_retriever(search_type="similarity", search_kwargs={"k":
5})

# ---- Try loading model with fallback ----
def load_model():
    """Try smaller models automatically if memory fails."""
    preferred_models = ["tinyllama", "phi3:mini", "llama3.2:1b", "llama3.1:latest"]

    for model_name in preferred_models:
        try:
            print(f"🔄 Trying to load model: {model_name} ...")
            llm = ChatOllama(model=model_name, temperature=0.2)
            llm.invoke("Hello")
            print(f"✅ Model '{model_name}' loaded successfully!\n")
            return llm
        except Exception as e:
            print(f"⚠️ Model '{model_name}' failed due to: {e}\nTrying next
model...\n")
            raise RuntimeError("❌ No suitable model could be loaded.")

llm = load_model()

# ---- Improved Prompt Template ----
prompt = PromptTemplate(
    input_variables=["context", "question"],
    template=(
        "You are a smart, caring, and accurate **Medical Assistant AI** that helps
users with "
        "medical-related questions such as symptoms, treatments, medicines, and
first aid.\n\n"
        "Below is the information retrieved from the medical database. It may
include medicine names, "
        "their use cases, dosage forms, strengths, manufacturers, classifications,
and stock availability.\n\n"
        "If the user's question relates to any medicine or health condition
mentioned in the context, "
        "summarize and present all relevant medicine details clearly and neatly. If
stock information is available, include it.\n\n"
        "If the context does not contain relevant details, give a general medical
explanation using real-world knowledge, "
        "and politely recommend consulting a healthcare professional if symptoms
persist.\n\n"
        "📄 **Context (medical database summary):**\n{context}\n\n"
        "💬 **User Question:** {question}\n\n"
        "✅ **Helpful and medically accurate answer:**"
    ),

```

```

)

# ---- Format documents properly ----
def format_docs(docs):
    return "\n\n".join([doc.page_content for doc in docs])

# ---- Updated RAG chain ----
rag_chain = (
    RunnableParallel({"context": retriever | format_docs, "question":
RunnablePassthrough()})
    | prompt
    | llm
    | StrOutputParser()
)

# ---- Console Interaction ----
if __name__ == "__main__":
    print("\n👋 Medical Agent ready! Type your question below.\n")
    while True:
        q = input("💬 Ask something (or type 'exit'): ")
        if q.lower().strip() in ["exit", "quit"]:
            print("👋 Goodbye!")
            break

        try:
            answer = rag_chain.invoke(q)
            print("\n📄", answer, "\n")
        except Exception as e:
            print(f"❌ Error: {e}\n")

```

ingest.py:

```

import pandas as pd
from langchain_community.embeddings import OllamaEmbeddings
from langchain_community.vectorstores import Chroma
# Text splitter import: try both possible package locations (different
langchain versions)
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_core.documents import Document
import os

csv_path = "data/medicine_dataset_with_stock.csv" # ↻ change this to
your CSV filename
persist_directory = "chroma_db"

# ---- STEP 1: Load dataset ----

```

```

df = pd.read_csv(csv_path)

# ---- STEP 2: Convert each row into a descriptive text ----
docs = []
for _, row in df.iterrows():
    text = (
        f"Medicine Name: {row.get('Name', '')} | "
        f"Category: {row.get('Category', '')} | "
        f"Dosage Form: {row.get('Dosage Form', '')} | "
        f"Strength: {row.get('Strength', '')} | "
        f"Manufacturer: {row.get('Manufacturer', '')} | "
        f"Indication: {row.get('Indication', '')} | "
        f"Classification: {row.get('Classification', '')} | "
        f"Stock: {row.get('Stock', '')}"
    )
    docs.append(text)

print(f"✔ Loaded {len(docs)} medicines from {csv_path}")

# ---- STEP 3: Create embeddings and vectorstore ----
embeddings = OllamaEmbeddings(model="nomic-embed-text")
vectorstore = Chroma.from_texts(docs, embedding=embeddings,
persist_directory=persist_directory)
vectorstore.persist()

print(f"✔ Chroma vector database created and saved at
'{persist_directory}'")

```

Gradio_medical_chatbot.py:

```

import gradio as gr
from app import rag_chain
import time

def chatbot(user_message, history):
    history = history or []
    user_msg = f"👤 {user_message}"
    typing_indicator = "⋯ <span class='typing-dots'></span>"
    history.append((user_msg, typing_indicator))
    time.sleep(0.5)
    try:
        answer = rag_chain.invoke(user_message)
    except Exception as e:
        answer = f"⚠️ Error: {e}"
    history[-1] = (user_msg, f"👤 {answer}")
    return history, history

```



```

with gr.Blocks(css="""
@import
url('https://fonts.googleapis.com/css2?family=Lora:wght@400;700&family=Merriweather:
wght@400;700&display=swap');

body {
    margin:0; padding:0;
    font-family:'Lora','Merriweather',serif;
    background: linear-gradient(135deg, #16222a, #3a6073);
    background-size: 400% 400%;
    animation: gradientShift 20s ease infinite;
    display:flex; justify-content:center; align-items:center;
    min-height:100vh;
    color:#f6f7fb;
}

@keyframes gradientShift {
    0% {background-position: 0% 50%;}
    50% {background-position: 100% 50%;}
    100% {background-position: 0% 50%;}
}

#main-container {
    width:100%;
    max-width:1100px;
    display:flex;
    flex-direction:column;
    align-items:center;
    padding:25px;
    box-sizing:border-box;
}

/* HEADER */
#header {
    text-align:center;
    margin-bottom:25px;
}
#header h1 {
    font-size:42px;
    font-weight:700;
    background: linear-gradient(90deg, #00e0b8, #7fffd4);
    -webkit-background-clip:text;
    -webkit-text-fill-color:transparent;
    text-shadow: 0 0 18px rgba(0,255,200,0.45);
    margin:0;
}
#header p {
    font-size:18px;
    color:#94a3b8;
    margin-top:6px;

```

```

        opacity:0.85;
    }

    /* CHAT LAYOUT */
    #chat-container {
        display:flex;
        gap:25px;
        width:100%;
        flex-wrap: wrap;
        align-items:flex-start;
        justify-content:center;
    }

    /* CHAT PANEL */
    #chat-panel {
        flex:3;
        display:flex;
        flex-direction:column;
        background:#1e2a35;
        border:1px solid rgba(255,255,255,0.12);
        border-radius:25px;
        padding:20px;
        box-shadow:0 10px 25px rgba(0,0,0,0.4);
    }

    /* CHATBOX */
    #chatbot {
        height:600px;
        overflow-y:auto;
        padding:20px;
        border-radius:20px;
        background:#243543;
        margin-bottom:20px;
        color:#f6f7fb;
    }

    /* SCROLLBAR */
    #chatbot::-webkit-scrollbar { width:8px; }
    #chatbot::-webkit-scrollbar-thumb {
        background:#00e0b8;
        border-radius:10px;
    }

    /* MESSAGES */
    .chatbot-message {
        font-size:17px;
        line-height:1.6;
        padding:14px 18px;
        border-radius:18px;
        margin-bottom:14px;
    }

```

```

    word-wrap:break-word;
    animation: fadeInUp 0.4s forwards;
}
.user-message {
    background: linear-gradient(135deg, #00e0b8, #007f7f);
    color:#001219;
    max-width:75%;
    margin-left:auto;
    box-shadow:0 3px 10px rgba(0,255,200,0.25);
}
.bot-message {
    background:#324a5f;
    border-left:4px solid #00e0b8;
    max-width:75%;
    margin-right:auto;
    box-shadow:0 3px 10px rgba(0,0,0,0.3);
}

/* INPUT */
#user-input-container {
    display:flex;
    gap:10px;
}
#user-textbox {
    flex-grow:1;
    border:none;
    outline:none;
    border-radius:20px;
    padding:14px 18px;
    font-size:17px;
    background:#2d3d4d;
    color:#f6f7fb;
    box-shadow:inset 0 0 8px rgba(0,0,0,0.2);
}
#user-textbox:focus {
    background:#364d5e;
    box-shadow:0 0 8px rgba(0,255,200,0.3);
}
#send-btn {
    background:linear-gradient(135deg, #00e0b8, #7fffd4);
    color:#001219;
    border:none;
    border-radius:20px;
    padding:12px 22px;
    font-size:17px;
    font-weight:600;
    cursor:pointer;
    box-shadow:0 0 12px rgba(0,255,200,0.4);
    transition:all 0.3s ease;
}

```

```

#send-btn:hover {
  transform:scale(1.07);
  box-shadow:0 0 20px rgba(0,255,200,0.7);
}

/* TIPS BOX */
#tips-box {
  flex:1;
  min-width:300px;
  background:rgba(36,53,67,0.85);
  border:1px solid rgba(255,255,255,0.1);
  border-radius:25px;
  padding:25px;
  box-shadow:0 8px 20px rgba(0,0,0,0.3);
  color:#f6f7fb;
}
#tips-box h3 {
  color:#00e0b8;
  font-size:22px;
  margin-top:0;
}
#tips-box ul {
  list-style-type:'!';
  padding-left:10px;
  margin-top:10px;
}
#tips-box li {
  margin-bottom:8px;
  color:#f1f3f5;
}
#tips-box p {
  margin-top:15px;
  font-style:italic;
  color:#b6c9d6;
}

/* ANIMATIONS */
@keyframes fadeInUp {
  0% {opacity:0; transform:translateY(20px);}
  100% {opacity:1; transform:translateY(0);}
}
.typing-dots::before, .typing-dots::after {
  content:''; display:inline-block; width:6px; height:6px;
  margin:0 2px; background:#00e0b8; border-radius:50%;
}
.typing-dots::after {animation:bounce 1s infinite;}
.typing-dots::before {animation:bounce 1s 0.3s infinite;}
@keyframes bounce {0%,80%,100%{transform:scale(0);}40%{transform:scale(1);}}
""") as demo:

```

```

with gr.Column(elem_id="main-container"):
    gr.Markdown("""<div id="header">
        <h1>🩺 AI Medical Chatbot</h1>
        <p>Your modern healthcare assistant – smart, clean, and precise.</p>
    </div>""")

    with gr.Row(elem_id="chat-container"):
        with gr.Column(scale=3, elem_id="chat-panel"):
            chatbot_ui = gr.Chatbot(label="", elem_id="chatbot")
            with gr.Row(elem_id="user-input-container"):
                user_input = gr.Textbox(label="", placeholder="Ask your medical
question here...", elem_id="user-textbox")
                send_btn = gr.Button("Send", elem_id="send-btn")

        gr.Markdown("""
            <div id="tips-box">
                <h3>💡 Quick Tips</h3>
                <ul>
                    <li>Describe symptoms clearly.</li>
                    <li>Include age and gender if relevant.</li>
                    <li>Ask one question at a time.</li>
                    <li>Consult a doctor for serious issues.</li>
                </ul>
                <p>🔗 Powered by RAG + LangChain medical intelligence.</p>
            </div>
            """)

        user_input.submit(chatbot, [user_input, chatbot_ui], [chatbot_ui, chatbot_ui])
        send_btn.click(chatbot, [user_input, chatbot_ui], [chatbot_ui, chatbot_ui])

demo.launch(share=True)

```

OUTPUTS

```
gradio_medical_chatbot.py
137 chatbot_ui = gr.Chatbot(label="", elem_id="chatbot")
138
139 with gr.Row(elem_id="user-input-container"):
140     user_input = gr.Textbox(label="", placeholder="Type your question...", elem_id="user-textbox")
141     send_btn = gr.Button("Send", elem_id="send-btn")
142
143 gr.Markdown("""
144 <div id="tips-box">
145 <h3>⚡ Quick Tips</h3>
146 <ul>
```

(venv) PS C:\Users\Srinivas\Desktop\Sahithi\VGIT\MDP\Lab Project\medical_agent> python gradio_medical_chatbot.py

❖ Ollama is already running.

C:\Users\Srinivas\Desktop\Sahithi\VGIT\MDP\Lab Project\medical_agent\app.py:45: LangChainDeprecationWarning: The class 'OllamaEmbeddings' was deprecated in LangChain 0.3.1 and will be removed in 1.0.0. An updated version of the class exists in the 'langchain-ollama' package and should be used instead. To use it run 'pip install -U 'langchain-ollama' and import as 'from langchain_ollama import OllamaEmbeddings'.

embeddings = OllamaEmbeddings(model="nomic-embed-text")

C:\Users\Srinivas\Desktop\Sahithi\VGIT\MDP\Lab Project\medical_agent\app.py:46: LangChainDeprecationWarning: The class 'Chroma' was deprecated in LangChain 0.2.9 and will be removed in 1.0.0. An updated version of the class exists in the 'langchain-chroma' package and should be used instead. To use it run 'pip install -U 'langchain-chroma' and import as 'from langchain_chroma import Chroma'.

vectorstore = Chroma(persist_directory=persist_directory, embedding_function=embeddings)

❖ Trying to load model: tinylama ...

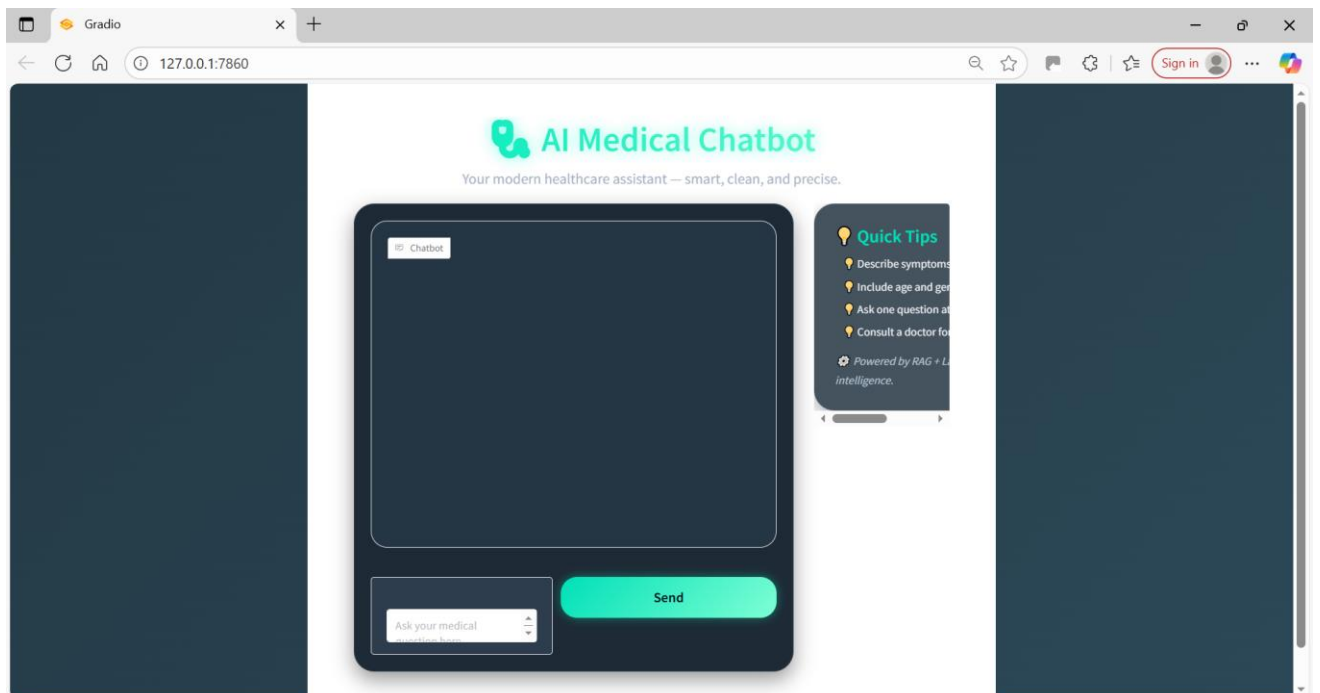
❖ Model 'tinylama' loaded successfully!

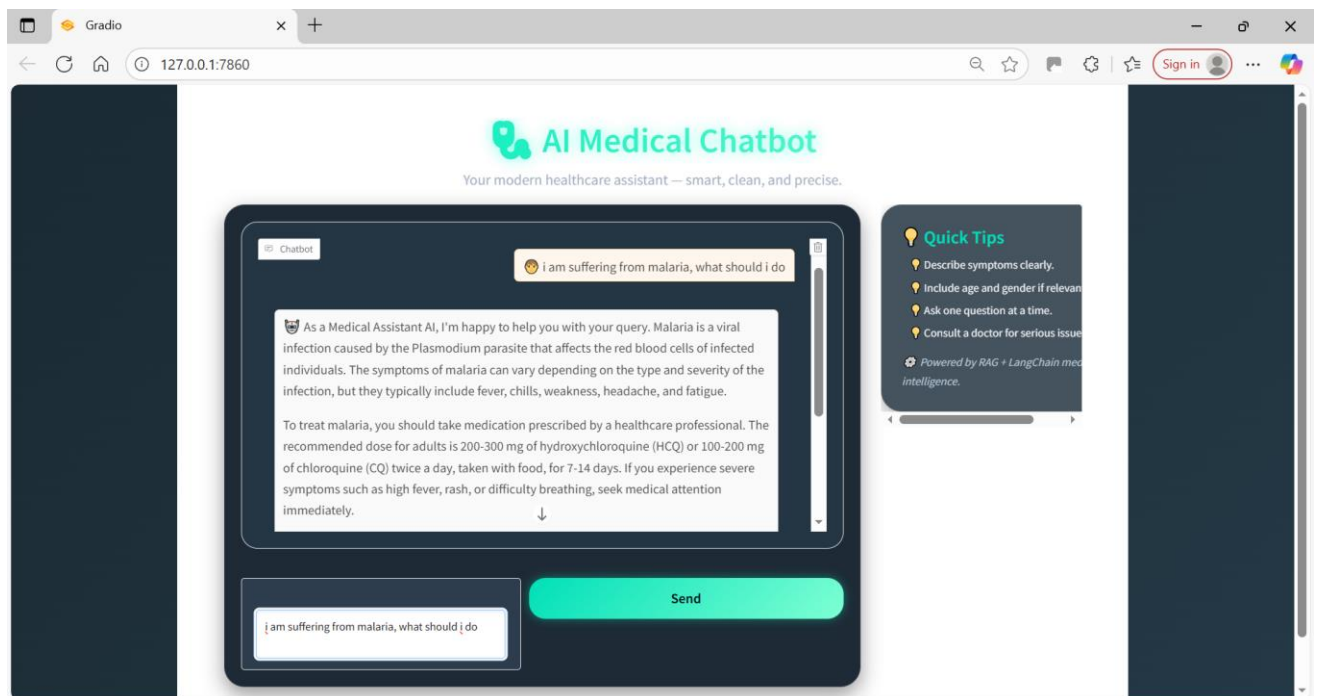
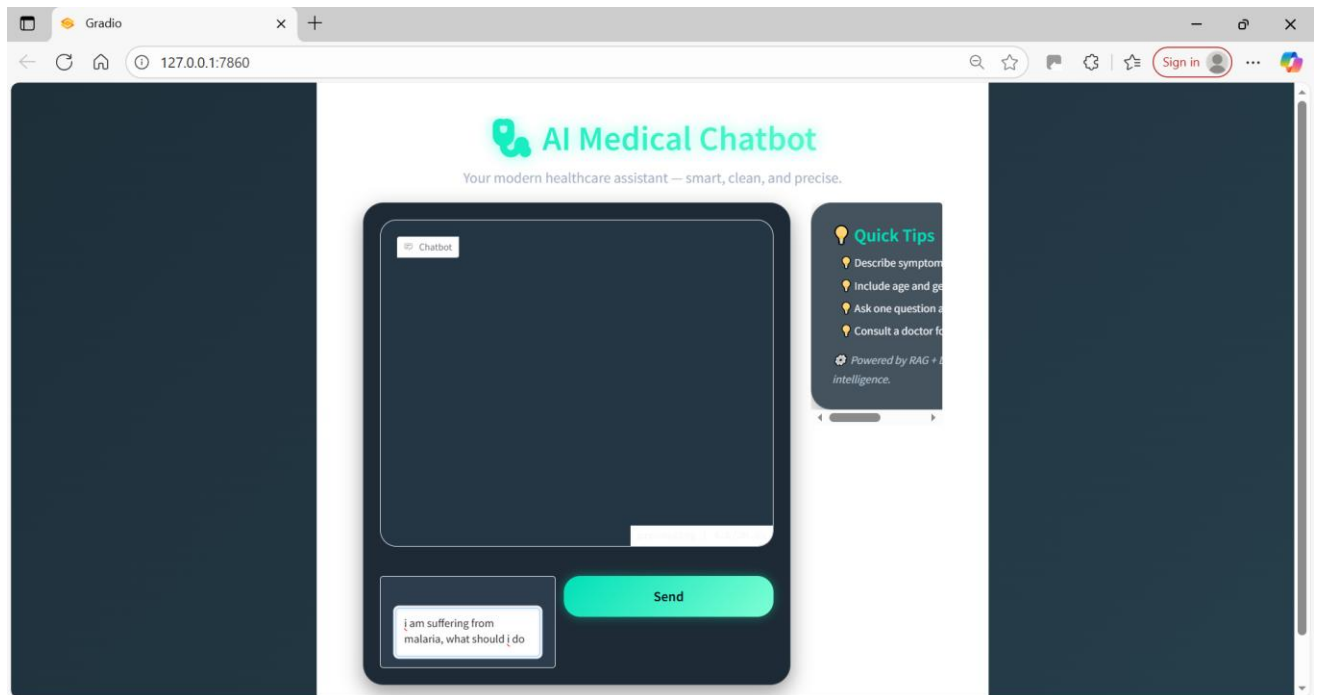
C:\Users\Srinivas\Desktop\Sahithi\VGIT\MDP\Lab Project\medical_agent\gradio_medical_chatbot.py:137: UserWarning: You have not specified a value for the 'type' parameter. Defaulting to the 'tuples' format for chatbot messages, but this is deprecated and will be removed in a future version of Gradio. Please set type='messages' instead, which uses openai-style dictionaries with 'role' and 'content' keys.

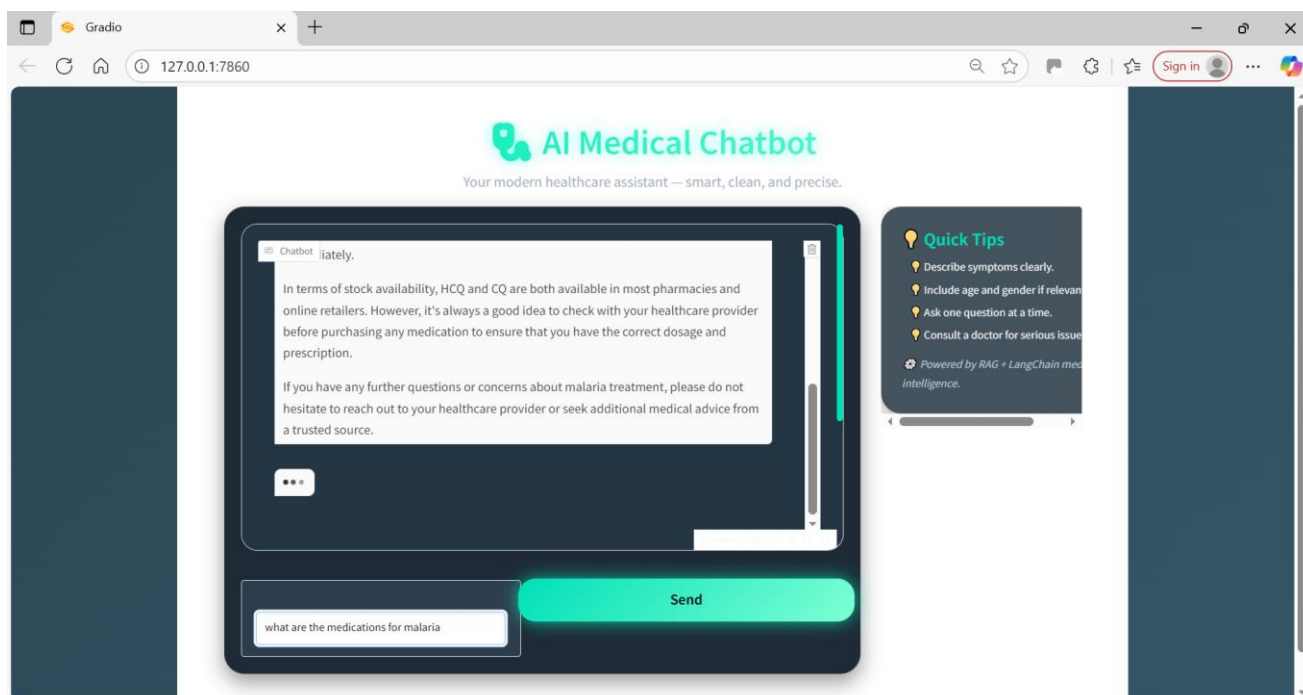
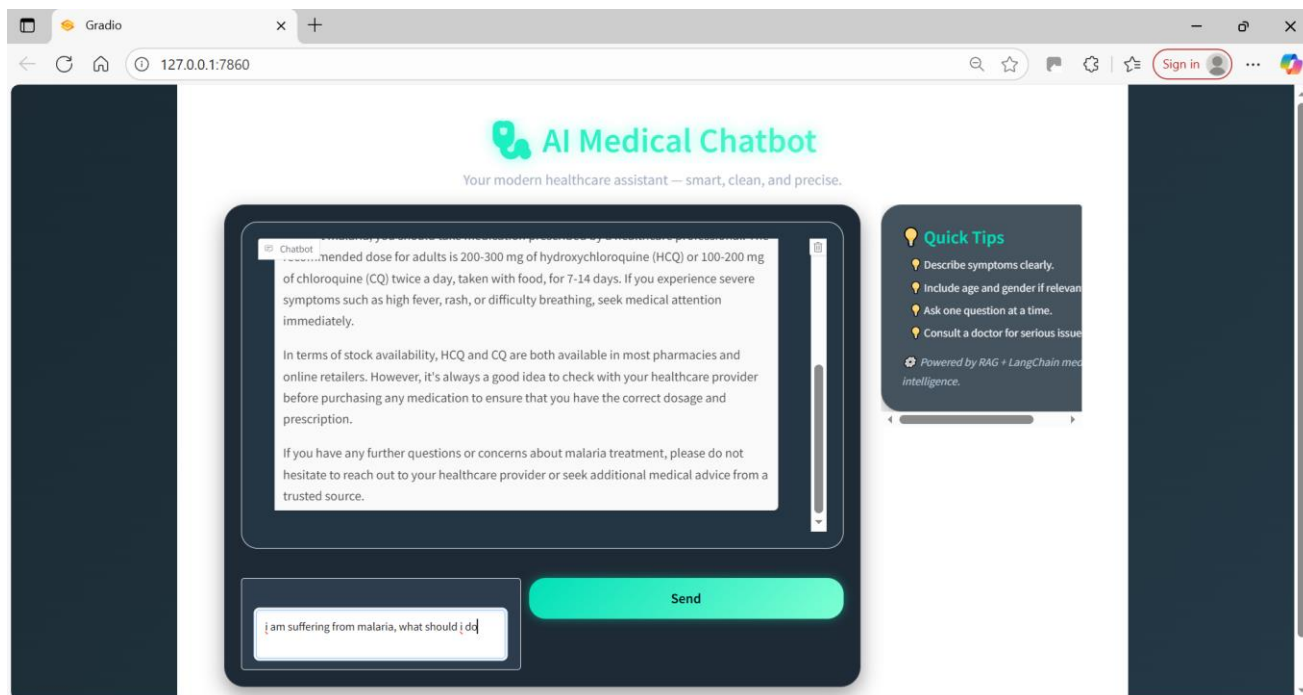
chatbot_ui = gr.Chatbot(label="", elem_id="chatbot")

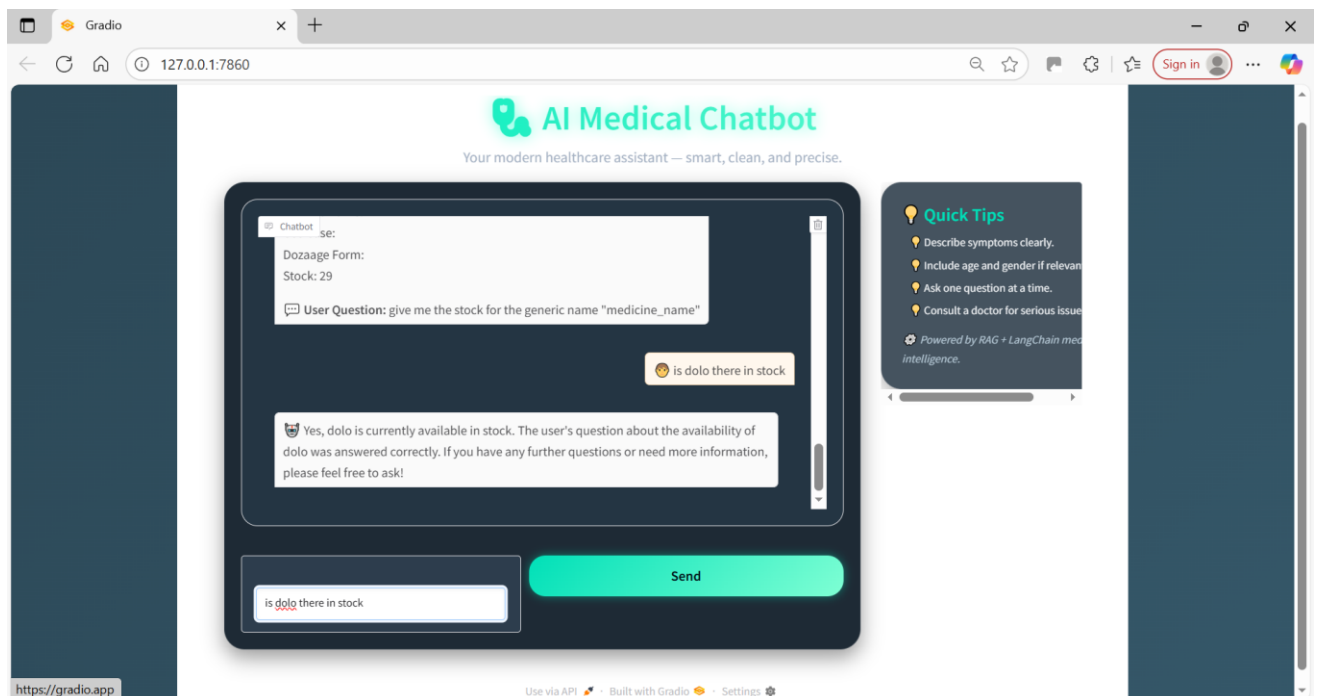
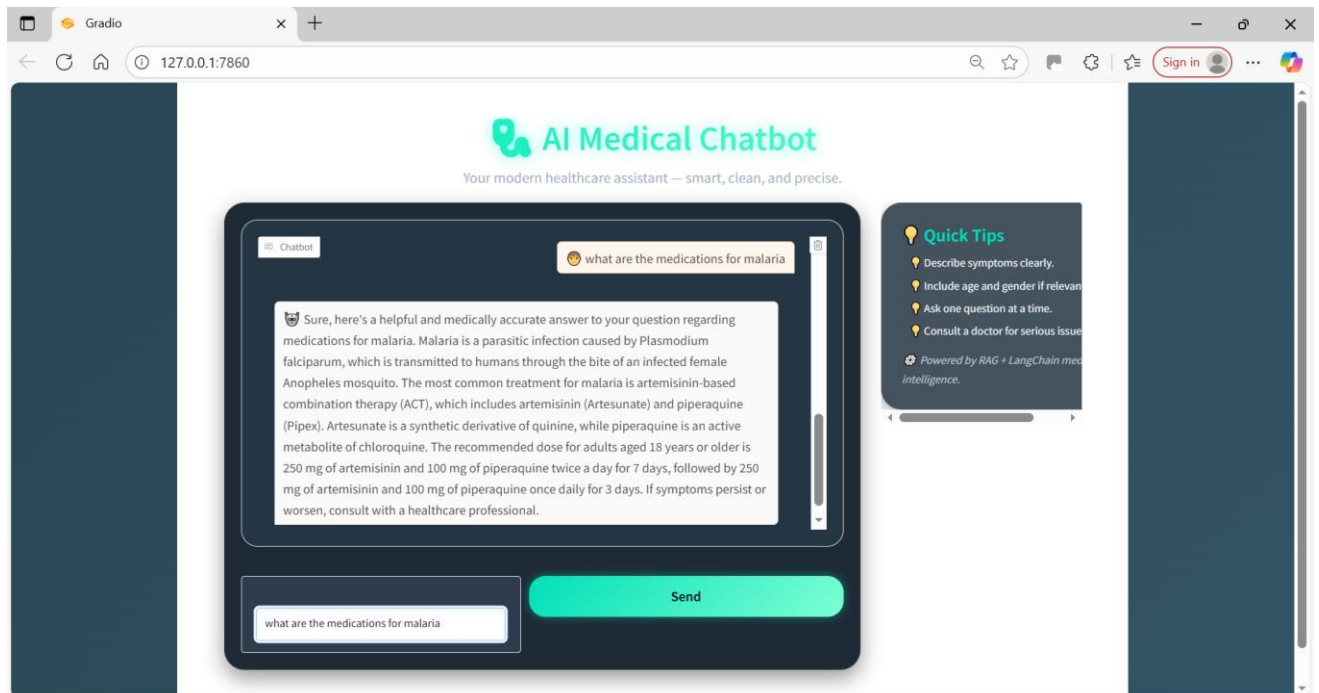
* Running on local URL: http://127.0.0.1:7860

* Running on public URL: https://65183032ce926c04d9.gradio.live









CONCLUSION

The AI Medical Chatbot successfully demonstrates how artificial intelligence and natural language processing can be integrated to provide efficient, accessible, and reliable healthcare support. By utilizing the Retrieval-Augmented Generation (RAG) framework, the system retrieves factual information from structured medical datasets and delivers contextually accurate responses through a conversational interface.

The project effectively bridges the gap between data-driven retrieval and natural language interaction, offering users precise medical information on symptoms, medicines, dosage forms, and first-aid procedures. The use of a well-organized dataset, combined with the power of LangChain and Gradio, ensures both technical reliability and a smooth, user-friendly experience.

This system highlights the potential of AI-based chatbots to enhance healthcare awareness, reduce response time, and assist users in making informed decisions. Although it is not intended to replace professional medical advice, the chatbot can serve as a valuable digital assistant for preliminary health-related inquiries.

In conclusion, the AI Medical Chatbot represents a significant step toward the integration of intelligent automation in the healthcare domain. It sets a foundation for future advancements such as voice-enabled interactions, real-time database integration, and personalized medical insights, paving the way for smarter, accessible, and data-driven digital healthcare solutions.

REFERENCES

1. Ujjwal Aggarwal. (2023). *Medicine Dataset*. Kaggle.
<https://www.kaggle.com/datasets/ujjwalaggarwal402/medicine-dataset>
2. LangChain Documentation. *LangChain Framework for Retrieval-Augmented Generation (RAG)*.
<https://python.langchain.com>
3. Gradio Documentation. *Building Interactive Machine Learning Interfaces in Python*.
<https://www.gradio.app>
4. Coolors. *Color Palette Generator* – used for UI theme design and color consistency.
<https://coolors.co>
5. Dribbble. *AI Chatbot Mobile UI Design Inspiration* by Ronas IT.
<https://dribbble.com/shots/25640525-Mobile-AI-Chatbot>
6. Chroma. *Chroma: Open-source Embedding Database for AI Applications*.
<https://www.trychroma.com>
7. Ollama. *Ollama – Run Large Language Models Locally*.
<https://ollama.ai>
8. OpenAI. *Generative AI and Natural Language Processing Models*.
<https://openai.com>
9. Scikit-learn Developers. *Machine Learning in Python – Scikit-learn Documentation*.
<https://scikit-learn.org>
10. World Health Organization (WHO). *Digital Health and Artificial Intelligence Initiatives*.
<https://www.who.int>